

Iterators & Generators in JavaScript

JavaScript-এ **Iterator** এবং **Generator** হলো এমন mechanism যগুলোর মাধ্যমে আমরা data-কে একবারে পুরোটা না নিয়ে একটি একটি করে, অর্থাৎ **lazily**, consume করতে পারি।

এই topic বুঝতে সবচেয়ে গুরুত্বপূর্ণ mental model হলো:

```
Iterable
  ↓
Symbol.iterator
  ↓
Iterator
  ↓
next()
  ↓
{ value, done }
```

আর Generator এই পুরো process-টাকে অনেক সহজ করে দেয়:

```
Generator Function
  ↓
generator object
  ↓
next()
  ↓
{ value, done }
```

1. Iterator কী?

Iterator হলো এমন একটি object যার মধ্যে একটি `next()` method থাকে।

`next()` call করলে এটি একটি object return করে:

```
{
  value: ...,
  done: ...
}
```

Example:

```
const iterator = {
  next() {
    return {
      value: 10,
      done: false
    };
  }
};

console.log(iterator.next());
```

Output:

```
{
  value: 10,
  done: false
}
```

Iterator-এর মূল requirement

একটি object iterator হতে হলে তার `next()` method থাকতে হবে।

```
const iterator = {
  next() {
    // return iterator result
  }
};
```

`next()` সাধারণত এই shape-এর object return করে:

```
{
  value: someValue,
  done: false
}
```

শেষ হয়ে গেলে:

```
{
  value: undefined,
  done: true
}
```

2. Iterator Protocol

JavaScript একটি নির্দিষ্ট protocol define করে যার মাধ্যমে iteration কাজ করে।

এটাকে বলা হয় **Iterator Protocol**।

Iterator-এর `next()` method প্রত্যেকবার একটি **Iterator Result Object** return করে।

```
{
  value: any,
  done: boolean
}
```

দুটি property:

Property	Meaning
<code>value</code>	বর্তমান value
<code>done</code>	iteration শেষ হয়েছে কি না

3. `next()` কীভাবে কাজ করে?

একটি simple iterator তৈরি করি:

```
const iterator = {
  current: 1,

  next() {
    if (this.current <= 3) {
      return {
        value: this.current++,
        done: false
      };
    }

    return {
      value: undefined,
      done: true
    };
  }
};
```

```
}  
};
```

এখন:

```
console.log(iterator.next());  
console.log(iterator.next());  
console.log(iterator.next());  
console.log(iterator.next());
```

Output:

```
{ value: 1, done: false }  
{ value: 2, done: false }  
{ value: 3, done: false }  
{ value: undefined, done: true }
```

4. `done: true` হওয়ার পর কী হয়?

Iterator শেষ হয়ে গেলে সাধারণত পরবর্তী `next()` call-গুলোও শেষ state return করে।

```
console.log(iterator.next());  
console.log(iterator.next());
```

Output:

```
{ value: undefined, done: true }  
{ value: undefined, done: true }
```

তবে custom iterator চাইলে অন্য behavior implement করতে পারে।

5. Iterable vs Iterator

এই দুইটা খুব গুরুত্বপূর্ণ।

Iterator

যার `next()` method আছে।

```
const iterator = {
  next() {
    return {
      value: 1,
      done: false
    };
  }
};
```

Iterable

যে object-এর মধ্যে:

```
Symbol.iterator
```

method থাকে এবং সেই method একটি iterator return করে।

```
const iterable = {
  [Symbol.iterator]() {
    return iterator;
  }
};
```

সহজভাবে:

```
Iterable
  |
  | Symbol.iterator()
  ↓
Iterator
  |
  | next()
  ↓
{ value, done }
```

6. Symbol.iterator

`Symbol.iterator` হলো JavaScript-এর একটি **well-known Symbol**।

এটি কোনো object কীভাবে iterate হবে সেটা define করার জন্য ব্যবহৃত হয়।

Example:

```
const numbers = [10, 20, 30];  
  
console.log(numbers[Symbol.iterator]);
```

Array-এর মধ্যে `Symbol.iterator` method built-in ভাবে আছে।

7. Array-এর Iterator

```
const numbers = [10, 20, 30];  
  
const iterator = numbers[Symbol.iterator]();  
  
console.log(iterator.next());  
console.log(iterator.next());  
console.log(iterator.next());  
console.log(iterator.next());
```

Output:

```
{ value: 10, done: false }  
{ value: 20, done: false }  
{ value: 30, done: false }  
{ value: undefined, done: true }
```

এখানে:

```
numbers
```

হলো **iterable**।

আর:

```
numbers[Symbol.iterator]()
```

return করে **iterator**।

8. Built-in Iterables

JavaScript-এর অনেক built-in data structure iterable।

Common examples:

- Array
- String
- Map
- Set
- TypedArray
- arguments object
- কিছু DOM collection

Example:

```
const text = "ABC";

const iterator = text[Symbol.iterator]();

console.log(iterator.next());
console.log(iterator.next());
console.log(iterator.next());
console.log(iterator.next());
```

Output:

```
{ value: "A", done: false }
{ value: "B", done: false }
{ value: "C", done: false }
{ value: undefined, done: true }
```

9. for...of কীভাবে কাজ করে?

আমরা সাধারণত লিখি:

```
const numbers = [10, 20, 30];

for (const number of numbers) {
```

```
    console.log(number);  
  }
```

কিন্তু internally conceptually ব্যাপারটা অনেকটা এরকম:

```
const iterator = numbers[Symbol.iterator]();  
  
let result = iterator.next();  
  
while (!result.done) {  
  console.log(result.value);  
  
  result = iterator.next();  
}
```

অর্থাৎ:

```
for...of  
  ↓  
Symbol.iterator  
  ↓  
iterator  
  ↓  
next()  
  ↓  
value
```

এই mental model খুব important ।

10. Custom Iterable তৈরি করা

আমরা নিজের object-কে iterable বানাতে পারি।

```
const numbers = {  
  start: 1,  
  end: 5,  
  
  [Symbol.iterator]() {  
    let current = this.start;  
    const end = this.end;  
  
    return {
```



```

    next() {
      if (current <= end) {
        return {
          value: current++,
          done: false
        };
      }

      return {
        value: undefined,
        done: true
      };
    }
  };
};

```

এখন:

```

for (const number of numbers) {
  console.log(number);
}

```

Output:

```

1
2
3
4
5

```

আমরা নিজের object-কে `for...of` compatible করে ফেলেছি।

11. Custom Iterable-এর Structure

একটি custom iterable সাধারণত এই pattern follow করে:

```

const iterable = {
  [Symbol.iterator]() {
    return {
      next() {
        return {

```

```

        value: ...,
        done: ...
    };
}
};
}
};

```

এখানে তিনটি layer:

```

iterable object
  ↓
Symbol.iterator()
  ↓
iterator object
  ↓
next()
  ↓
{ value, done }

```

12. Iterable এবং Iterator একই object-ও হতে পারে

একটি object একই সাথে iterable এবং iterator হতে পারে।

```

const counter = {
  current: 1,

  next() {
    if (this.current <= 3) {
      return {
        value: this.current++,
        done: false
      };
    }

    return {
      value: undefined,
      done: true
    };
  },

  [Symbol.iterator]() {
    return this;
  }
}

```

```
}  
};
```

এখন:

```
for (const value of counter) {  
  console.log(value);  
}
```

Output:

```
1  
2  
3
```

কারণ:

```
counter[Symbol.iterator]()
```

নিজেকেই return করছে।

13. Generator কী?

Generator হলো JavaScript-এর এমন একটি special function যা execution-কে pause এবং resume করতে পারে।

Generator function লিখতে:

```
function* numbers() {  
  yield 1;  
  yield 2;  
  yield 3;  
}
```

এখানে:

```
function*
```

দিয়ে generator function declare করা হয়েছে।

14. yield

Generator-এর সবচেয়ে গুরুত্বপূর্ণ keyword হলো:

```
yield
```

`yield` একটি value return করার পাশাপাশি generator-এর execution **pause** করে।

```
function* numbers() {  
  yield 10;  
  yield 20;  
  yield 30;  
}
```

Generator call করলে function-এর body সঙ্গে সঙ্গে execute হয় না।

```
const generator = numbers();
```

এখন:

```
console.log(generator);
```

এটি একটি **Generator Object**।

15. Generator-এর next()

```
function* numbers() {  
  yield 10;  
  yield 20;  
  yield 30;  
}  
  
const generator = numbers();  
  
console.log(generator.next());  
console.log(generator.next());  
console.log(generator.next());  
console.log(generator.next());
```

Output:

```
{ value: 10, done: false }  
{ value: 20, done: false }  
{ value: 30, done: false }  
{ value: undefined, done: true }
```

অর্থাৎ generator নিজেই একটি iterator ।

16. Generator Execution Flow

এই code দেখো:

```
function* test() {  
  console.log("A");  
  
  yield 1;  
  
  console.log("B");  
  
  yield 2;  
  
  console.log("C");  
}  
  
const generator = test();  
  
console.log("X");  
  
console.log(generator.next());  
  
console.log("Y");  
  
console.log(generator.next());  
  
console.log("Z");
```

Execution:

```
X  
A  
{ value: 1, done: false }
```

```
Y
B
{ value: 2, done: false }
Z
```

Generator function call করার সময়:

```
test()
```

সঙ্গে সঙ্গে body execute হয় না।

প্রথম:

```
generator.next()
```

call করলে execution শুরু হয় এবং প্রথম `yield` পর্যন্ত যায়।

17. Generator হলো State Machine-এর মতো

Generator-এর execution state মনে রাখতে পারে।

```
function* steps() {
  yield "Step 1";
  yield "Step 2";
  yield "Step 3";
}
```

এটা conceptually:

```
Start
  ↓
yield Step 1
  ↓
yield Step 2
  ↓
yield Step 3
  ↓
Done
```

প্রতিবার `next()` call করলে state পরের অবস্থায় যায়।

18. Generator-এর return value

Generator-এর মধ্যে `return` ব্যবহার করা যায়।

```
function* test() {  
  yield 1;  
  yield 2;  
  return 100;  
}
```

এখন:

```
const generator = test();  
  
console.log(generator.next());  
console.log(generator.next());  
console.log(generator.next());
```

Output:

```
{ value: 1, done: false }  
{ value: 2, done: false }  
{ value: 100, done: true }
```

খেয়াল করো:

```
return 100
```

এর `result`-এর `done` হলো `true`।

19. `yield` বনাম `return`

<code>yield</code>	<code>return</code>
Generator pause করে	Generator শেষ করে
<code>done: false</code>	<code>done: true</code>
Multiple <code>yield</code> করা যায়	সাধারণত final result

yield

return

পরে execution resume হয় আর resume হয় না

Example:

```
function* demo() {  
  yield 1;  
  yield 2;  
  return 3;  
}
```

Result:

```
1 → false  
2 → false  
3 → true
```

20. Generator দিয়ে for...of

Generator iterable হওয়ায় সরাসরি for...of করা যায়।

```
function* numbers() {  
  yield 10;  
  yield 20;  
  yield 30;  
}  
  
for (const number of numbers()) {  
  console.log(number);  
}
```

Output:

```
10  
20  
30
```


21. Generator-এর বড় সুবিধা: Lazy Evaluation

Generator data একবারে তৈরি করে memory-তে রাখে না।

Example:

```
function* numbers() {  
  yield 1;  
  yield 2;  
  yield 3;  
}
```

এখানে:

```
numbers()
```

call করলেই 1, 2, 3 সব তৈরি হয়ে যায় না।

যখন:

```
next()
```

call করবে, তখন প্রয়োজন অনুযায়ী পরের value তৈরি হবে।

এটাকে বলা হয়:

Lazy evaluation

22. Eager vs Lazy

Eager

```
const numbers = [];  
  
for (let i = 1; i <= 1000000; i++) {  
  numbers.push(i);  
}
```

এখানে এক million values আগে তৈরি হচ্ছে।

Lazy

```
function* numbers() {  
  let i = 1;  
  
  while (true) {  
    yield i++;  
  }  
}
```

এখানে সব number memory-তে আগে তৈরি হচ্ছে না।

যখন প্রয়োজন:

```
generator.next();
```

তখন value তৈরি হচ্ছে।

23. Infinite Sequence with Generator

Generator-এর অন্যতম powerful use case হলো **infinite sequence** তৈরি করা।

Example:

```
function* infiniteNumbers() {  
  let number = 1;  
  
  while (true) {  
    yield number++;  
  }  
}
```

এটি theoretically কখনো শেষ হবে না।

```
const numbers = infiniteNumbers();  
  
console.log(numbers.next());  
console.log(numbers.next());  
console.log(numbers.next());  
console.log(numbers.next());
```

Output:

```
{ value: 1, done: false }  
{ value: 2, done: false }  
{ value: 3, done: false }  
{ value: 4, done: false }
```

24. Infinite Generator সরাসরি `for...of` করলে বিপদ

এটা করো না:

```
for (const number of infiniteNumbers()) {  
  console.log(number);  
}
```

কারণ loop কখনো শেষ হবে না।

এটি continuously চলবে।

Infinite generator ব্যবহার করার সময় **termination condition** দিতে হবে।

25. Infinite Sequence থেকে প্রথম N values

```
function* infiniteNumbers() {  
  let number = 1;  
  
  while (true) {  
    yield number++;  
  }  
}  
  
const numbers = infiniteNumbers();  
  
for (let i = 0; i < 5; i++) {  
  console.log(numbers.next().value);  
}
```

Output:

```
1
2
3
4
5
```

26. Fibonacci Sequence with Generator

Generator দিয়ে Fibonacci sequence খুব সুন্দরভাবে implement করা যায়।

```
function* fibonacci() {
  let a = 0;
  let b = 1;

  while (true) {
    yield a;

    [a, b] = [b, a + b];
  }
}
```

ব্যবহার:

```
const fib = fibonacci();

for (let i = 0; i < 10; i++) {
  console.log(fib.next().value);
}
```

Output:

```
0
1
1
2
3
5
8
13
```

21
34

27. Generator দিয়ে Custom Range

আমরা এমন range তৈরি করতে পারি যা lazy:

```
function* range(start, end, step = 1) {  
  for (let current = start; current <= end; current += step) {  
    yield current;  
  }  
}
```

ব্যবহার:

```
for (const number of range(1, 10, 2)) {  
  console.log(number);  
}
```

Output:

```
1  
3  
5  
7  
9
```

28. Reverse Range

```
function* reverseRange(start, end) {  
  for (let current = start; current >= end; current--) {  
    yield current;  
  }  
}  
  
for (const number of reverseRange(5, 1)) {
```

```
    console.log(number);  
  }
```

Output:

```
5  
4  
3  
2  
1
```

29. Generator এবং `Symbol.iterator`

Generator-এর সবচেয়ে interesting ব্যাপার হলো:

Generator object নিজেই **iterable** এবং **iterator**।

```
function* numbers() {  
  yield 1;  
  yield 2;  
}  
  
const generator = numbers();  
  
console.log(generator[Symbol.iterator]() === generator);
```

Output:

```
true
```

অর্থাৎ generator:

```
Iterable  
+  
Iterator
```

দুটোই।

30. Generator দিয়ে Custom Iterable সহজ করা

আগে custom iterable বানাতে অনেক boilerplate লাগত:

```
const iterable = {
  [Symbol.iterator]() {
    let current = 1;

    return {
      next() {
        if (current <= 3) {
          return {
            value: current++,
            done: false
          };
        }
      }

      return {
        value: undefined,
        done: true
      };
    }
  };
};
```

Generator দিয়ে একই কাজ:

```
const iterable = {
  *[Symbol.iterator]() {
    yield 1;
    yield 2;
    yield 3;
  }
};
```

এখন:

```
for (const value of iterable) {
  console.log(value);
}
```

Output:

```
1  
2  
3
```

Generator custom iterable তৈরির কাজ অনেক সহজ করে দেয়।

31. Generator Method Syntax

Object-এর method হিসেবেও generator লেখা যায়।

```
const obj = {  
  *numbers() {  
    yield 1;  
    yield 2;  
    yield 3;  
  }  
};
```

ব্যবহার:

```
const generator = obj.numbers();  
  
console.log(generator.next());
```

32. Class-এর মধ্যে Generator Method

```
class Counter {  
  *numbers() {  
    yield 1;  
    yield 2;  
    yield 3;  
  }  
}  
  
const counter = new Counter();  
  
for (const value of counter.numbers()) {
```



```
    console.log(value);  
  }
```

33. Generator-এ next(value)

Generator-এর `next()` method-এ value পাঠানো যায়।

এটা advanced কিন্তু খুব গুরুত্বপূর্ণ feature।

```
function* test() {  
  const value = yield "Give me a value";  
  
  console.log("Received:", value);  
}  
  
const generator = test();  
  
console.log(generator.next());  
console.log(generator.next(100));
```

প্রথম:

```
generator.next()
```

generator-কে প্রথম `yield` পর্যন্ত নিয়ে যায়।

তারপর:

```
generator.next(100)
```

এর `100` value গিয়ে assign হয়:

```
const value = yield "Give me a value";
```

তাই:

```
Received: 100
```

34. গুরুত্বপূর্ণ: প্রথম `next(value)` `value` কোথায় যায়?

এই বিষয়টা interview-এ আসতে পারে।

```
const generator = test();  
  
generator.next(100);
```

এখানে `100` প্রথম `yield`-এ পাঠানো যায় না, কারণ `generator` তখনও শুরু হয়নি।

সাধারণ pattern:

```
generator.next();  
generator.next(value);
```

35. Generator দিয়ে Data Communication

Generator শুধু value produce করে না; `next(value)` দিয়ে বাইরে থেকেও data receive করতে পারে।

```
function* calculator() {  
  const a = yield "Enter first number";  
  const b = yield "Enter second number";  
  
  return a + b;  
}  
  
const calc = calculator();  
  
console.log(calc.next());  
console.log(calc.next(10));  
console.log(calc.next(20));
```

Final result:

```
{  
  value: 30,  
  done: true  
}
```

এটি generator-এর advanced coroutine-like behavior-এর ভিত্তি।

36. `yield*` — Generator Delegation

এক generator অন্য iterable বা generator-এর values yield করতে পারে।

```
function* first() {  
  yield 1;  
  yield 2;  
}  
  
function* second() {  
  yield* first();  
  
  yield 3;  
  yield 4;  
}
```

এখন:

```
for (const value of second()) {  
  console.log(value);  
}
```

Output:

```
1  
2  
3  
4
```

`yield*` মানে conceptually:

"এই iterable/generator-এর values-গুলোও yield করো।"

37. `yield*` দিয়ে Array delegation

```
function* numbers() {  
  yield* [10, 20, 30];  
}
```

```
}  
  
for (const number of numbers()) {  
  console.log(number);  
}
```

Output:

```
10  
20  
30
```

38. Custom Iterable + `for...of`

ধরি আমাদের একটি custom range object দরকার:

```
const range = {  
  start: 1,  
  end: 5,  
  
  *[Symbol.iterator]() {  
    for (let i = this.start; i <= this.end; i++) {  
      yield i;  
    }  
  }  
};
```

এখন:

```
for (const number of range) {  
  console.log(number);  
}
```

Output:

```
1  
2  
3
```

4
5

এটাই modern JavaScript-এ custom iterable implement করার clean pattern ।

39. Other Syntax যেগুলো **Iterables** ব্যবহার করে

শুধু `for...of` নয়, আরও অনেক JavaScript feature iterable protocol ব্যবহার করে ।

Array spread

```
const numbers = new Set([1, 2, 3]);  
  
const array = [...numbers];  
  
console.log(array);
```

Array destructuring

```
const [first, second] = new Set([10, 20, 30]);  
  
console.log(first);  
console.log(second);
```

`Array.from()`

```
const set = new Set([1, 2, 3]);  
  
const array = Array.from(set);  
  
console.log(array);
```

এই operations-এর পেছনে iterable protocol গুরুত্বপূর্ণ ভূমিকা রাখে ।

40. `for...in` vs `for...of`

এটা খুব ভালোভাবে মনে রাখতে হবে ।

for...in

Object-এর enumerable property keys iterate করে।

```
const user = {  
  name: "Shariar",  
  age: 31  
};  
  
for (const key in user) {  
  console.log(key);  
}
```

Output:

```
name  
age
```

for...of

Iterable-এর values iterate করে।

```
const numbers = [10, 20, 30];  
  
for (const number of numbers) {  
  console.log(number);  
}
```

Output:

```
10  
20  
30
```

Summary

for...in	for...of
Keys/properties	Values
Enumerable properties	Iterable values
Objects-এর সাথে common	Arrays/Strings/Sets/Maps ইত্যাদির সাথে common

for...in

for...of

Symbol.iterator প্রয়োজন নেই Iterable protocol ব্যবহার করে

41. কেন সাধারণ **Object** for...of করা যায় না?

```
const user = {  
  name: "Sharir",  
  age: 31  
};  
  
for (const value of user) {  
  console.log(value);  
}
```

Error হবে:

TypeError: user is not iterable

কারণ সাধারণ object-এর built-in:

Symbol.iterator

নেই।

42. Object-কে Iterable বানানো

```
const user = {  
  name: "Sharir",  
  age: 31,  
  
  *[Symbol.iterator]() {  
    yield this.name;  
    yield this.age;  
  }  
};
```

এখন:

```
for (const value of user) {  
  console.log(value);  
}
```

Output:

```
Shariar  
31
```

43. Iterator-এর একটি গুরুত্বপূর্ণ Rule

`Symbol.iterator()` method-এর result অবশ্যই iterator-এর মতো object হওয়া উচিত।

অর্থাৎ সেখানে `next()` থাকতে হবে।

ভুল:

```
const obj = {  
  [Symbol.iterator]() {  
    return {};  
  }  
};
```

সঠিক:

```
const obj = {  
  [Symbol.iterator]() {  
    return {  
      next() {  
        return {  
          value: 1,  
          done: true  
        };  
      }  
    };  
  }  
};
```

Generator ব্যবহার করলে এই boilerplate অনেক কমে যায়।

44. Iterator-এর State

Iterator সাধারণত stateful।

Example:

```
const iterator = [10, 20, 30][Symbol.iterator]();

console.log(iterator.next());
console.log(iterator.next());
```

Output:

```
{ value: 10, done: false }
{ value: 20, done: false }
```

একই iterator-এর পরের `next()` আগের অবস্থার ওপর depend করে।

45. দুইটি আলাদা Iterator

```
const numbers = [10, 20, 30];

const iterator1 = numbers[Symbol.iterator]();
const iterator2 = numbers[Symbol.iterator]();

console.log(iterator1.next());
console.log(iterator1.next());

console.log(iterator2.next());
```

Output:

```
{ value: 10, done: false }
{ value: 20, done: false }
{ value: 10, done: false }
```

কারণ প্রত্যেকবার নতুন iterator তৈরি হয়েছে।

46. Iterator Reusability

Iterable সাধারণত নতুন iterator তৈরি করতে পারে:

```
const numbers = [1, 2, 3];

for (const n of numbers) {
  console.log(n);
}

for (const n of numbers) {
  console.log(n);
}
```

দুইবারই:

```
1
2
3
```

কিন্তু একই iterator reuse করলে:

```
const iterator = numbers[Symbol.iterator]();

for (const value of iterator) {
  console.log(value);
}

for (const value of iterator) {
  console.log(value);
}
```

দ্বিতীয় loop-এ আর values নাও থাকতে পারে, কারণ iterator ইতোমধ্যে exhausted ।

47. Iterator Closing

`for...of` loop মাঝপথে `break` করলে JavaScript iterator-এর `return()` method থাকলে সেটি call করতে পারে ।

Custom iterator-এ cleanup করার জন্য `return()` implement করা যায় ।

```

const iterable = {
  [Symbol.iterator]() {
    let current = 1;

    return {
      next() {
        if (current <= 5) {
          return {
            value: current++,
            done: false
          };
        }

        return {
          done: true
        };
      },

      return() {
        console.log("Iterator closed");
        return {
          done: true
        };
      }
    };
  }
};

```

এখন:

```

for (const value of iterable) {
  console.log(value);

  if (value === 2) {
    break;
  }
}

```

Cleanup hook হিসেবে `return()` কাজ করতে পারে।

48. Generator-এর `return()`

Generator-এরও `return()` method আছে।

```
function* numbers() {
  try {
    yield 1;
    yield 2;
    yield 3;
  } finally {
    console.log("Cleanup");
  }
}

const generator = numbers();

console.log(generator.next());

generator.return();

console.log(generator.next());
```

`return()` generator-এর execution terminate করতে পারে।

`finally` block cleanup-এর জন্য execute হতে পারে।

49. Generator-এর `throw()`

Generator-এর মধ্যে বাইরে থেকে error inject করা যায়।

```
function* test() {
  try {
    yield 1;
  } catch (error) {
    console.log("Caught:", error.message);
  }
}

const generator = test();

console.log(generator.next());

generator.throw(new Error("Something went wrong"));
```

এটি advanced generator control flow।

50. Generator কখন ব্যবহার করব?

Generator useful হতে পারে:

- lazy sequences
- large datasets
- infinite sequences
- custom iterables
- pagination
- streaming-like processing
- stateful iteration
- parser/tokenizer
- data pipelines
- step-by-step computation

51. Generator বনাম Normal Function

Normal Function	Generator Function
<code>function</code>	<code>function*</code>
সাধারণত একবার return	Multiple <code>yield</code>
execution শেষ হয়	execution pause/resume করতে পারে
value directly return করে	Generator object return করে
lazy sequence তৈরি করা কঠিন	lazy sequence সহজ
<code>return</code>	<code>yield</code> + <code>return</code>

52. Generator বনাম Iterator

Iterator manually implement করা যায়:

```
const iterator = {
  current: 1,

  next() {
    return {
      value: this.current++,
      done: this.current > 4
    };
  }
};
```

```
}  
};
```

Generator একই concept সহজ করে:

```
function* numbers() {  
  yield 1;  
  yield 2;  
  yield 3;  
}
```

Mental model

```
Iterator  
= protocol  
  
Generator  
= iterator তৈরি করার convenient mechanism
```

Generator নিজেই iterator protocol follow করে।

53. Real-world Example: Pagination

ধরি API থেকে page-by-page data আসছে।

Conceptually:

```
async function* paginate(fetchPage) {  
  let page = 1;  
  
  while (true) {  
    const data = await fetchPage(page);  
  
    if (data.length === 0) {  
      return;  
    }  
  
    yield data;  
  
    page++;  
  }  
}
```

```
}  
}
```

ব্যবহার:

```
for await (const page of paginate(fetchPage)) {  
  console.log(page);  
}
```

এখানে আমরা **Async Generator** ব্যবহার করছি।

Async Generator JavaScript-এর asynchronous iteration-এর অংশ। এটি পরে Async JavaScript গভীরভাবে শেখার সময় আরও বিস্তারিতভাবে দেখা যায়।

54. Generator দিয়ে Lazy Data Pipeline

Generator ব্যবহার করে data pipeline বানানো যায়।

```
function* numbers() {  
  for (let i = 1; i <= 10; i++) {  
    yield i;  
  }  
}  
  
function* evenNumbers(iterable) {  
  for (const number of iterable) {  
    if (number % 2 === 0) {  
      yield number;  
    }  
  }  
}  
  
for (const number of evenNumbers(numbers())) {  
  console.log(number);  
}
```

Output:

```
2  
4  
6
```

```
8  
10
```

এখানে সব intermediate data array হিসেবে তৈরি করার প্রয়োজন নেই।

55. Generator দিয়ে Lazy Filtering

```
function* filter(iterable, callback) {  
  for (const value of iterable) {  
    if (callback(value)) {  
      yield value;  
    }  
  }  
}
```

ব্যবহার:

```
const numbers = [1, 2, 3, 4, 5, 6];  
  
for (const number of filter(numbers, n => n % 2 === 0)) {  
  console.log(number);  
}
```

Output:

```
2  
4  
6
```

56. Generator দিয়ে Lazy Mapping

```
function* map(iterable, callback) {  
  for (const value of iterable) {  
    yield callback(value);  
  }  
}
```

ব্যবহার:


```
const numbers = [1, 2, 3];

const doubled = map(numbers, n => n * 2);

for (const number of doubled) {
  console.log(number);
}
```

Output:

```
2
4
6
```

57. Lazy Pipeline

এখন আমরা combine করতে পারি:

```
function* map(iterable, callback) {
  for (const value of iterable) {
    yield callback(value);
  }
}

function* filter(iterable, callback) {
  for (const value of iterable) {
    if (callback(value)) {
      yield value;
    }
  }
}

const numbers = [1, 2, 3, 4, 5, 6];

const doubled = map(numbers, n => n * 2);

const result = filter(doubled, n => n > 5);

for (const value of result) {
  console.log(value);
}
```

Output:

```
6
8
10
12
```

এখানে computation lazy।

58. Generator Infinite Sequence + Limit

Infinite sequence এবং lazy processing combine করা যায়।

```
function* numbers() {
  let n = 1;

  while (true) {
    yield n++;
  }
}

function* take(iterable, count) {
  let i = 0;

  for (const value of iterable) {
    if (i++ >= count) {
      return;
    }

    yield value;
  }
}
```

ব্যবহার:

```
for (const value of take(numbers(), 5)) {
  console.log(value);
}
```

Output:

```
1
2
3
4
5
```

এখানে infinite generator হলেও consumer মাত্র 5টি value নিচ্ছে।

59. Iterable Protocol vs Iterator Protocol

দুটো আলাদা করে মনে রাখো।

Iterable Protocol

Object-এর:

```
[Symbol.iterator]()
```

থাকতে হবে।

এটি একটি iterator return করবে।

Iterator Protocol

Returned iterator-এর:

```
next()
```

থাকতে হবে।

এবং:

```
next()
```

return করবে:

```
{
  value,
```

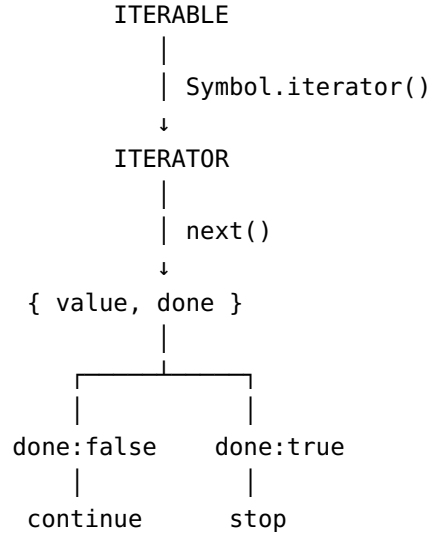
```
    done
  }
```

Complete chain

```
Object
  ↓
Symbol.iterator()
  ↓
Iterator
  ↓
next()
  ↓
{ value, done }
```

60. Most Important Mental Model

এই পুরো topic-এর সবচেয়ে important flow:



Generator হলো:

```
function*
  ↓
generator object
  ↓
```

```
next()
  ↓
yield
  ↓
{ value, done }
```

61. Common Mistakes

Mistake 1: Iterator আর Iterable একই মনে করা

```
iterator.next();
```

এটা iterator-এর feature ।

```
iterable[Symbol.iterator]();
```

এটা iterable-এর protocol ।

Mistake 2: সাধারণ Object `for...of` করা

```
const user = {
  name: "John"
};

for (const value of user) {
  console.log(value);
}
```

এটি কাজ করবে না ।

Object-কে iterable বানাতে হবে ।

Mistake 3: Infinite generator-এ unlimited loop

```
function* numbers() {
  let n = 1;
```

```
while (true) {  
  yield n++;  
}
```

এর ওপর:

```
for (const n of numbers()) {  
  console.log(n);  
}
```

করলে loop শেষ হবে না।

Mistake 4: Generator call করলেই body execute হবে মনে করা

```
function* test() {  
  console.log("Hello");  
  yield 1;  
}  
  
test();
```

"Hello" সঙ্গে সঙ্গে print হবে না।

```
const generator = test();  
  
generator.next();
```

করলে execution শুরু হবে।

Mistake 5: প্রথম next(value) -এ value পাঠানো

```
generator.next(100);
```

প্রথম call-এর value কোনো yield expression-এ পাওয়া যায় না, কারণ generator তখনও execution শুরু করেনি।

Mistake 6: `yield` এবং `return` confuse করা

```
yield
```

pause করে।

```
return
```

generator শেষ করে।

62. Performance এবং Memory

Generator-এর সবচেয়ে বড় advantage:

Lazy computation

ধরি:

```
function* numbers() {  
  for (let i = 0; i < 10000000000; i++) {  
    yield i;  
  }  
}
```

এখানে এক billion number-এর array বানানো হচ্ছে না।

শুধু প্রয়োজনের সময়:

```
generator.next();
```

পরবর্তী value produce হবে।

তাই generators বিশেষভাবে useful হতে পারে যখন:

- data অনেক বড়
- সব data একসাথে প্রয়োজন নেই
- stream-like processing দরকার
- infinite sequence দরকার

তবে generator নিজে সবসময় faster নয়। এর মূল benefit হলো **lazy control flow** এবং **memory-efficient processing pattern**।

63. Interview Question: Iterable কী?

Answer:

যে object-এর মধ্যে:

```
Symbol.iterator
```

method থাকে এবং সেই method একটি iterator return করে, তাকে iterable বলা হয়।

64. Interview Question: Iterator কী?

Answer:

যে object-এর `next()` method থাকে এবং `next()` একটি iterator result object:

```
{  
  value,  
  done  
}
```

return করে, সেটি iterator।

65. Interview Question: `Symbol.iterator` কী?

`Symbol.iterator` হলো JavaScript-এর একটি well-known Symbol, যা কোনো object-এর iteration behavior define করে।

66. Interview Question: `for...of` কীভাবে কাজ করে?

Conceptually:


```
const iterator = iterable[Symbol.iterator]();

let result = iterator.next();

while (!result.done) {
  // use result.value

  result = iterator.next();
}
```

অর্থাৎ `for...of` iterable protocol ব্যবহার করে।

67. Interview Question: Generator কী?

Generator হলো special function যেটি:

```
function*
```

syntax ব্যবহার করে এবং `yield` এর মাধ্যমে execution pause/resume করতে পারে।

Generator function call করলে generator object পাওয়া যায়, যা iterable এবং iterator দুটোই।

68. Interview Question: Generator কেন useful?

Common use cases:

- Lazy evaluation
 - Infinite sequences
 - Large data processing
 - Custom iterables
 - Pagination
 - Data pipelines
 - Stateful iteration
 - Streaming-like workflows
-

69. Interview Question: Generator কি iterable?

হ্যাঁ।

```
function* numbers() {  
  yield 1;  
  yield 2;  
}  
  
const generator = numbers();  
  
console.log(  
  generator[Symbol.iterator]() === generator  
);
```

Output:

```
true
```

70. Output Prediction Practice

Question 1

```
function* test() {  
  yield 10;  
  yield 20;  
}  
  
const generator = test();  
  
console.log(generator.next());  
console.log(generator.next());  
console.log(generator.next());
```

নিজে output বের করার চেষ্টা করো।

Question 2

```
function* test() {  
  console.log("A");  
  
  yield 1;  
  
  console.log("B");
```

```
    yield 2;
  }

  const generator = test();

  console.log("X");
  console.log(generator.next());
  console.log("Y");
  console.log(generator.next());
```

Execution order কী হবে?

Question 3

```
const obj = {
  *[Symbol.iterator]() {
    yield "A";
    yield "B";
    yield "C";
  }
};

for (const value of obj) {
  console.log(value);
}
```

Output কী?

Question 4

```
function* numbers() {
  let n = 1;

  while (true) {
    yield n++;
  }
}

const generator = numbers();

console.log(generator.next().value);
```

```
console.log(generator.next().value);  
console.log(generator.next().value);
```

Output কী?

71. Practice Tasks

Level 1 — Basic Iterator

নিজে একটি iterator বানাও যেটি:

```
10  
20  
30  
40  
50
```

একটি একটি করে return করবে।

Level 2 — Custom Iterable

একটি object বানাও:

```
const range = {  
  start: 1,  
  end: 10  
};
```

এটাকে iterable বানাও যাতে:

```
for (const n of range) {  
  console.log(n);  
}
```

Output:

```
1  
2  
3
```

```
...  
10
```

Level 3 — Generator Range

এমন generator বানাও:

```
range(5, 10)
```

যা produce করবে:

```
5  
6  
7  
8  
9  
10
```

Level 4 — Infinite Generator

এমন generator বানাও যা infinite even numbers produce করবে:

```
2  
4  
6  
8  
10  
...
```

তারপর প্রথম 10টি value বের করো।

Level 5 — Fibonacci Generator

Generator দিয়ে Fibonacci sequence implement করো।

প্রথম 20টি value print করো।

Level 6 — Lazy Filter

Generator দিয়ে এমন function বানাও:

```
filter(iterable, callback)
```

যেটা নতুন array তৈরি না করে lazy ভাবে filtered values yield করবে।

Level 7 — take()

এমন function বানাও:

```
take(iterable, count)
```

যা যেকোনো iterable থেকে প্রথম `count` values yield করবে।

তারপর combine করো:

```
take(infiniteNumbers(), 10)
```

72. Quick Revision

Iterator

```
iterator.next();
```

returns:

```
{  
  value,  
  done  
}
```

Iterable

```
iterable[Symbol.iterator]()
```

একটি iterator return করে।

Generator

```
function* test() {  
  yield 1;  
  yield 2;  
}
```

Generator object

```
const generator = test();
```

Get next value

```
generator.next();
```

Iterate

```
for (const value of generator) {  
  console.log(value);  
}
```

Infinite generator

```
function* numbers() {  
  let n = 1;  
  
  while (true) {  
    yield n++;  
  }  
}
```

Custom iterable

```
const obj = {
  *[Symbol.iterator]() {
    yield 1;
    yield 2;
    yield 3;
  }
};
```

73. Final Cheat Sheet

Concept	Key Point
Iterator	<code>next()</code> method থাকে
<code>next()</code>	<code>{ value, done }</code> return করে
<code>done</code>	iteration শেষ হয়েছে কিনা
Iterable	<code>Symbol.iterator</code> থাকে
<code>Symbol.iterator</code>	iterator return করে
<code>for...of</code>	iterable protocol ব্যবহার করে
Generator	<code>function*</code> দিয়ে তৈরি
<code>yield</code>	pause + value produce
<code>return</code>	generator শেষ করে
<code>next(value)</code>	generator-এ value পাঠাতে পারে
<code>yield*</code>	অন্য iterable/generator delegate করে
Infinite generator	endless lazy sequence
Generator object	Iterable + Iterator
Lazy evaluation	প্রয়োজনের সময় value তৈরি

74. Final Mental Model

এই topic-টা মনে রাখার সবচেয়ে সহজ উপায়:


```

ITERABLE
  |
  | Symbol.iterator()
  ↓
ITERATOR
  |
  | next()
  ↓
{ value, done }

```

আর Generator:

```

function*
  |
  ↓
Generator Object
  |
  |— next()
  |   ↓
  |   yield value
  |
  |— Symbol.iterator()
  |   ↓
  |   itself
  |
  |— yield*
  |   ↓
  |   delegate iteration

```

সবশেষে এই relationship-টা মাথায় রাখো:

Iterable বলে "আমাকে কীভাবে **iterate** করবে সেটা আমি জানি।"

Iterator বলে "next value কী সেটা আমি জানি।"

Generator হলো **iterator** বানানোর **powerful** ও **convenient way**।

`for...of` **iterable**-এর `Symbol.iterator()` নিয়ে **iterator** পায় এবং `next()` **call** করে একে একে **values consume** করে।

এই চারটা relationship clear হলে **Iterators & Generators**-এর **core concept** প্রায় পুরোপুরি **clear** হয়ে যাবে।